Engineering Deep Dive & Viva Preparation Guide — Day 3 (2026-10-13)

Project Title: VERBAMIND — AI-Assisted Placement Communication Coaching Platform
Target Audience: B.Tech AI/ML Final Year Student, Project Guide, and External Viva Examiners
Document Type: Architectural Deep Dive, DSP & Acoustic Pipeline Analysis, and Comprehensive Viva Defense Guide
Milestones Covered: Milestone 3 (Acoustic Metrics, Physical Pause Segmentation, Delivery Measurements Panel, Clickable Gap Playback, and Synchronized Interactive Transcript Playback)

Table of Contents

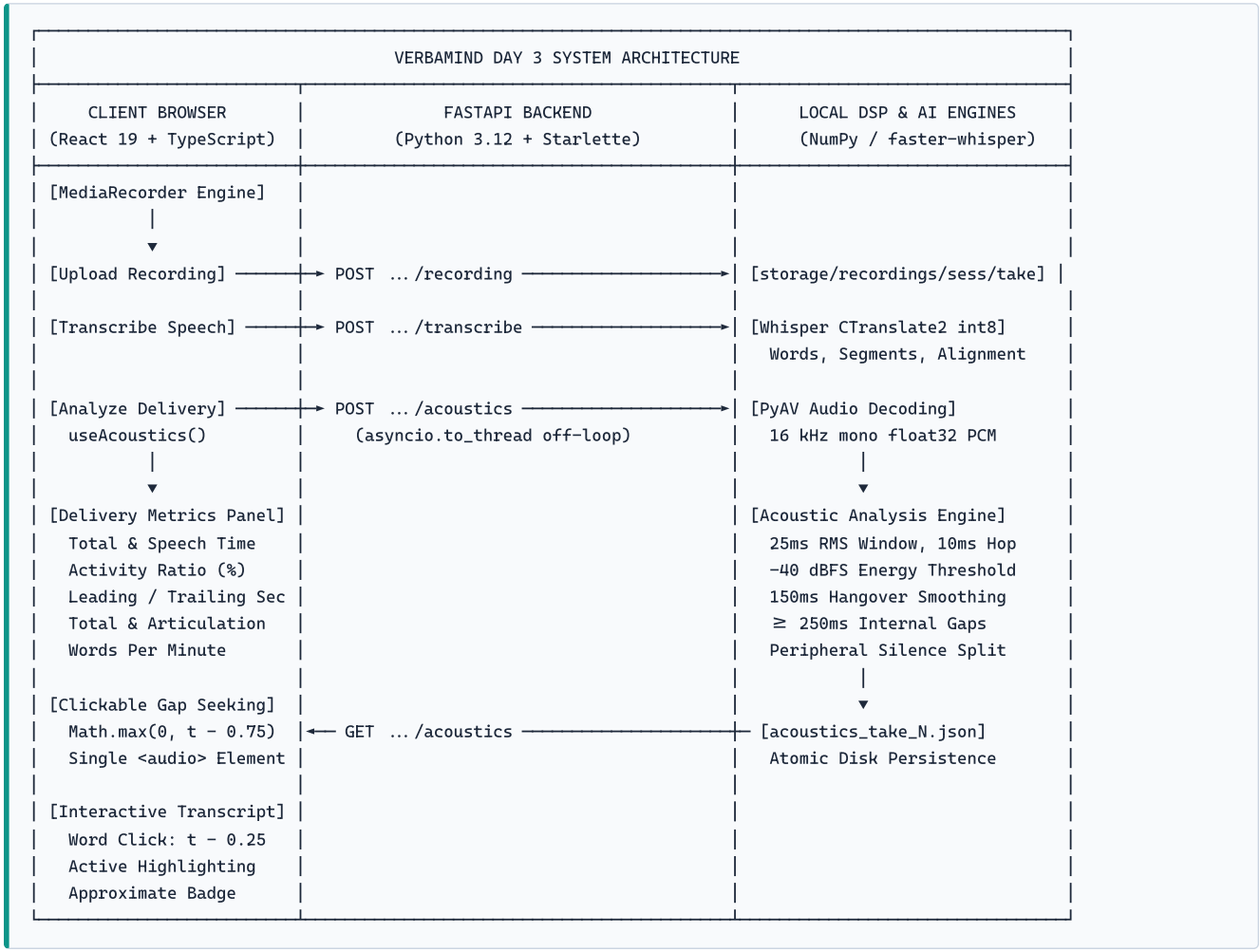
1. Executive Overview: What Was Built & The Day 3 Evolution
2. Codebase Architecture & Responsibilities of Changed Files
3. Acoustic Digital Signal Processing (DSP) Engine Deep Dive
4. Acoustic API Endpoints, Off-Event-Loop Processing & Persistence
5. Frontend Delivery UI, Clickable Gap Seeking & Decimal Timestamps
6. Interactive Synchronized Transcript Playback Engine
7. Physical Silence vs Perceived Pause: Waveform Energy Investigation
8. Scientific & Pedagogical Invariants: Avoiding Pseudoscientific Traps
9. Pretrained Foundation ASR vs Future Trained Acoustic Fluency Model
10. Security Defenses, Boundary Guards & Production Limitations
11. Automated Verification Evidence & Comprehensive Test Matrix
12. 22 Deep Technical Viva Defense Questions & Answers
13. Live Evaluation Demo Script & Conference Paper Roadmap

---

1. Executive Overview: What Was Built & The Day 3 Evolution

The Progression of VERBAMIND

- **Day 1**: Established the client-side browser voice recording studio (`MediaRecorder` API), visualizer, mic permission lifecycle, and mock exercise catalogue. Audio remained stranded in browser memory.
- **Day 2**: Constructed the server bridge with private local audio upload (`POST /api/sessions/{id}/recording`), magic-byte container validation, in-process PyAV audio decoding (WebM/Opus to 16 kHz float32 PCM), and local offline speech transcription using `faster-whisper` on CPU via an asynchronous worker queue.
- **Day 3**: Integrated the **Acoustic Delivery Analysis Engine** and **Interactive Feedback UI**:
- **Physical Waveform Analysis**: Implemented pure Python/NumPy Short-Time Energy Root Mean Square (RMS) frame analysis detecting active speech intervals, speech activity ratios, and separating peripheral (leading/trailing) latency from communicative speech.
- **Internal Non-Speech Gap Segmentation**: Implemented Goldman-Eisler psycholinguistic segmentation ($\ge 250\text{ ms}$) with a 150 ms hangover window to bridge unvoiced stop consonant closures (/p/, /t/, /k/) without fragmenting lexical speech.
- **Off-Event-Loop FastAPI Endpoint**: Added `POST` and `GET /api/sessions/{session_id}/recordings/{take_number}/acoustics` , executing heavy NumPy DSP off the event loop via `asyncio.to_thread` , and caching results atomically to private disk.
- **Browser Delivery Measurements Panel**: Added a modern, high-contrast "Audio Delivery Measurements" card in `AudioPlayer.tsx` showing duration, active speech, percentage, silence breakdown, and dual-rate WPM (total pace and articulation pace).
- **Clickable Gap Seeking with Lead-In**: Rendered detected gaps as accessible buttons. Clicking a gap seeks the player to 0.75s before gap onset (`Math.max(0, start - 0.75)`) and triggers playback so users can hear the preceding context and pause boundary.
- **Precise Decimal Seconds**: Formatted gap breakdown timestamps with integer-hundredths arithmetic (`mm:ss.SS`), ensuring short pauses never display identical start and end values.
- **Interactive Synchronized Transcript Playback**: Tokenized Whisper cross-attention word timestamps into interactive buttons. Clicking any word seeks to 0.25s before that word's onset and starts playback; playback real-time active highlighting highlights spoken words sequentially.
- **Strict Scientific Invariants**: Explicitly labeled alignments as approximate cross-attention; separated acoustic metrics from linguistic WER; disclaimed mock scores; and prevented overlapping audio streams by strictly constraining playback to a single HTML5 audio element.



2. Codebase Architecture & Responsibilities of Changed Files

Every file in the repository satisfies a strict separation of concerns:

File Path	Layer / Technology	Primary Responsibility & Day 3 Additions
`backend/src/backend/services/acoustics.py`	DSP / Domain Logic	Pure Python/NumPy frame-energy RMS speech detector, hangover smoothing, peripheral silence separation, internal gap extraction ($\ge 250\text{ms}$), dual WPM calculation, and zero-division guardrails.
`backend/src/backend/api/v1/endpoints/acoustics.py`	FastAPI Controller	Exposes `POST` and `GET` acoustic analysis endpoints. Enforces loopback security, dev environment boundary, session path sanitization, off-event-loop execution (`asyncio.to_thread`), and atomic JSON persistence.
`backend/src/backend/api/v1/api.py`	API Router	Registers acoustics router under `/sessions/{session_id}/recordings/{take_number}/acoustics` .
`backend/src/backend/cli/compare_models.py`	CLI Tool	Offline ASR benchmark comparing `base.en` and `small.en` on identical audio with dynamic programming Levenshtein WER alignment and RTF measurement.
`frontend/src/hooks/useAcoustics.ts`	React State Hook	Manages acoustic lifecycle (`idle` , `analyzing` , `completed` , `failed`). Uses `activeTakeRef` to reject stale asynchronous responses when takes change.
`frontend/src/components/AudioPlayer.tsx`	UI Component	Renders the Audio Delivery Measurements panel, clickable gap seeking (`Math.max(0, start - 0.75)`), precise decimal timestamps (`mm:ss.SS`), interactive transcript word buttons (`Math.max(0, start - 0.25)`), real-time `.transcript-word-active` highlighting, and academic integrity disclaimers.
`frontend/src/index.css`	Styling	CSS styling for acoustic badges, delivery cards, decimal gap buttons, interactive word buttons, active highlight glow, and focus accessibility rings.
`backend/tests/test_acoustics.py`	Backend Tests	12 unit tests verifying energy RMS calculation, pure silence, continuous speech, empty inputs, hangover window bridging, gap filtering, and dual WPM derivation.
`backend/tests/test_acoustics_api.py`	Backend Tests	8 integration tests verifying loopback origin defense, development environment guard, session path traversal rejection, persistence lifecycle, and pure silence responses.
`frontend/src/hooks/useAcoustics.test.ts`	Frontend Tests	6 tests verifying hook state transitions, error handling, take-switching stale response rejection, and reset behavior.
`frontend/src/components/AudioPlayer.test.tsx`	Frontend Tests	10 integration tests verifying upload, transcription, acoustic measurement display, decimal gap timestamps (`00:05.01 – 00:05.29`), clickable gap seeking, interactive word clicking, active highlighting, and take reset.

3. Acoustic Digital Signal Processing (DSP) Engine Deep Dive

3.1 Framing and Windowing

Audio input is decoded via PyAV to 16,000 Hz 32-bit floating-point mono PCM ($x[n] \in [-1.0, 1.0]$). To extract energy over time, the signal is divided into overlapping frames: - **Frame Duration**: 25.0 ms ($N = 400$ samples at 16 kHz). - **Hop Duration**: 10.0 ms ($H = 160$ samples at 16 kHz). - **Frame Rate**: $100\text{ frames per second}$, giving 10 ms temporal granularity.

For frame k (starting at sample index $k \cdot H$): $$\text{RMS}*k = \sqrt{\frac{1}{N}} \sum*{m=0}^{N-1} (x[k \cdot H + m])^2$$

3.2 Decibels Full Scale (dBFS)

Audio energy is converted to logarithmic decibels relative to full scale: $$\text{dBFS}*k = 20 \log*{10}(\text{RMS}*k + 10^{-7})$$ The epsilon 10^{-7} prevents $\log*{10}(0)$ arithmetic errors during digital silence, bounding the minimum decibel floor to -140.0 dBFS.

3.3 Calibrated Speech Energy Threshold

- **Threshold**: -40.0 dBFS.

- A frame is tentatively active if $\text{dBFS}_k \ge -40.0\text{ dBFS}$.
- Ambient background noise in standard home/office microphone setups typically rests between -55 dBFS and -50 dBFS. Conversational speech energy peaks between -28 dBFS and -16 dBFS. The -40.0 dBFS cutoff sits reliably above the noise floor while retaining soft vowel tails and unvoiced fricatives.

3.4 The Hangover Window Smoothing Algorithm

Unvoiced stop consonants in English (/p/, /t/, /k/) involve complete vocal tract closure for 30 ms to 80 ms before releasing. Without temporal smoothing, a naive threshold fragments continuous words into disjoint acoustic bursts. - **Hangover Window**: 150 ms (15 frames). - When frame energy drops below -40.0 dBFS, the detector maintains active status for 15 frames. If another speech burst occurs within 150 ms, the intervening dip is bridged seamlessly as continuous speech. - **Minimum Speech Burst**: 80 ms (8 frames). Transient acoustic pulses shorter than 80 ms (such as mechanical mic clicks or desk thumps) are rejected as non-speech.

3.5 Goldman-Eisler Psycholinguistic Gap Segmentation

Internal pauses are identified as intervals between speech bursts: - **Minimum Gap Duration**: 250 ms (0.25 s). - In psycholinguistic literature (Goldman-Eisler, 1968), pauses below 250 ms represent articulatory transitions and stop closures inherent to phonetic execution. Pauses $\ge 250\text{ ms}$ represent communicative, syntactic, and cognitive intervals (breathing, clause boundaries, emphasis, or lexical retrieval). - Only intervals $\ge 250\text{ ms}$ are classified into `internal_gaps`.

3.6 Peripheral Silence Separation

User recordings inherently contain peripheral delays: 1. **Leading Silence**: From $t = 0.0$ until the first detected speech frame (user prepares to speak after clicking Record). 2. **Trailing Silence**: From the end of the last detected speech frame until the end of the recording (delay before clicking Stop). 3. **Internal Non-Speech Gaps**: Silence intervals occurring strictly between the first and last speech boundaries. Conflating peripheral silence with internal speech pauses would severely corrupt speech activity ratios and pace calculations.

3.7 Dual Speaking Rate Derivation

Speaking rate requires a recognised transcript and is labeled as `rate_is_transcript_dependent = True`: 1. **Overall Speaking Pace (Total WPM)**: $$\text{WPM}_{\text{total}} = \frac{\text{Word Count}}{\text{Total Duration (seconds)} / 60.0}$$ *Measures conversational rhythm, including all pauses and breathing. Normal range:* ***120 – 160 WPM****. 2.* ***Articulation Rate (Speech-Only WPM)****:* $$\text{WPM}_{\text{speech}} = \frac{\text{Word Count}}{\text{Detected Speech Duration (seconds)} / 60.0}$$ Measures motor speech production tempo discounting pauses. Normal range: **140 – 190 WPM**. *(A speaker with 119 Total WPM and 138 Articulation WPM speaks articulately at 138 WPM but uses deliberate, natural pauses between sentences).*

---

4. Acoustic API Endpoints, Off-Event-Loop Processing & Persistence

4.1 Off-Event-Loop Execution (`asyncio.to_thread`)

Audio decoding and frame-by-frame RMS computation are CPU-bound operations. Running them directly in a FastAPI `async def` route would block the event loop, freezing concurrent API requests and health probes. The endpoint dispatches processing to worker threads:

```
def _process_audio_acoustics() → AcousticMeasurements:
    audio_array = decode_audio(recording_path)
    # Load transcript if available to derive WPM
    transcript = load_transcript_if_present()
    return analyze_acoustics(audio_array, transcript_word_count=transcript.word_count)

measurements = await asyncio.to_thread(_process_audio_acoustics)
```

4.2 Atomic Persistence

Acoustic measurements are saved to private session storage as `acoustics_take_{take_number}.json`. The write is atomic: it writes to a `.tmp` file and replaces the destination using `os.replace`, guaranteeing that partial writes never corrupt disk state. Subsequent calls to `GET … /acoustics` serve the cached JSON instantly without re-decoding audio.

---

5. Frontend Delivery UI, Clickable Gap Seeking & Decimal Timestamps

5.1 The Audio Delivery Measurements Card

When delivery analysis completes, `AudioPlayer.tsx` displays: - **Recording Duration vs Speech-Active Duration**: E.g. `47.2s total • 41.0s active speech`. - **Speech Activity Ratio**: E.g. `87.0% active speech ratio`. - **Peripheral Silence**: E.g. `0.44s leading delay • 0.70s trailing delay`. - **Speaking Pace**: `120 WPM speaking pace` with `138 WPM articulation rate` subtitle. - **Internal Non-Speech Gaps List**: Expandable breakdown listing each detected pause interval.

5.2 Clickable Gap Seeking with Surrounding Speech Context

Listening to silence in isolation provides no diagnostic value to a learner. To understand why a pause occurred, the student must hear the words leading up to the pause. - Clicking a gap button executes: `typescript const targetSeek = Math.max(0, gap.start_seconds − 0.75); audioRef.current.currentTime = targetSeek; audioRef.current.play();` - The 0.75s lead-in allows the user to hear the preceding lexical context, experience the pause, and hear the following continuation. - The `Math.max(0, …)` boundary prevents negative seek values for early gaps.

5.3 Precise Decimal Timestamps Formatting

Rounding timestamps to integer seconds caused short pauses to display identical start and end values (e.g. `[00:05 – 00:05]`). We implemented `formatTimestampPrecise` with integer-hundredths arithmetic:

```
function formatTimestampPrecise(seconds: number): string {
  const totalHundredths = Math.round(seconds * 100);
  const totalSeconds = Math.floor(totalHundredths / 100);
  const hundredths = totalHundredths % 100;
  const minutes = Math.floor(totalSeconds / 60);
  const remainingSeconds = totalSeconds % 60;
  return `${String(minutes).padStart(2, '0')}:${String(remainingSeconds).padStart(2, '0')}.${String(hundredths).padStart(2,
}
```

Short pauses now render distinct, unambiguous timestamps (e.g. `[00:05.01 – 00:05.29]`, duration 0.28s).

6. Interactive Synchronized Transcript Playback Engine

6.1 Tokenized Word-Level Seeking

Whisper cross-attention weights generate word timestamps (`seg.words`). In `AudioPlayer.tsx`: - When word timestamps exist, each word renders as an accessible `<button className="transcript-word-btn">`. - Clicking a word calls `handlePlayTimestamp(w.start)`. - It seeks the player to 0.25s before word onset: `typescript const targetSeek = Math.max(0, w.start − 0.25); audioRef.current.currentTime = targetSeek; audioRef.current.play();` - The 0.25s lead-in ensures the user hears the initial phonetic consonant release rather than an abrupt mid-vowel onset.

6.2 Real-Time Playback Highlighting

The audio element emits `timeupdate` events as playback advances:

```
const isWordActive = currentTime ≥ w.start && currentTime ≤ w.end;
```

The active word receives `.transcript-word-active`, lighting up with an accent highlight. When playback leaves the interval, the class is removed.

6.3 Graceful Degradation & Fallback Strategy

1. **Word-Level Available**: Renders individual clickable word buttons.
2. **Words Missing (Segment Only)**: Renders segment phrases as clickable buttons with `.transcript-segment-active`.
3. **No Timestamps**: Safely falls back to plain text `<p className="transcript-recognised-text">`.

6.4 Single Player Invariant (Zero Overlapping Streams)

Both gap playback and transcript word seeking operate exclusively on the single `<audio ref={audioRef}>` element. No secondary `AudioContext` or audio elements are spawned, mathematically guaranteeing zero overlapping audio playback.

6.5 Safe Text Rendering & Accessibility

- Text is rendered strictly as React text nodes (`{w.word}` , `{seg.text}`), preventing XSS injection.
- Buttons include descriptive `aria-label` attributes and keyboard focus rings (`Enter` and `Space` operable).
- The header displays an `"Approximate Alignment"` badge, clearly informing the user that Whisper alignments are approximate cross-attention navigation aids, not phoneme-level ground truth.

---

7. Physical Silence vs Perceived Pause: Waveform Energy Investigation

During Day 3 live testing on recording `take_2.webm` (13.44s), an essential acoustic phenomenon was investigated and recorded in `docs/07-testing/test-evidence-day-3.md` :

7.1 The Acoustic Discrepancy

- Whisper marked the lexical end of the word `"audio"` at **7.76s**.
- The next phrase began at **9.16s**.
- A human listener perceives a pause of $\approx 1.4\text{s}$.
- However, the acoustic detector reported a gap of **0.715s** (**8.445s to 9.160s**).

7.2 Waveform Energy Breakdown

Inspecting the raw float32 samples revealed: 1. **7.76s to 8.41s**: The speaker released their breath after the word `"audio"` . This post-vocalic breath and vocal tract tail produced energy between -27.4 dBFS and -24.3 dBFS (above the -40.0 dBFS speech threshold). The detector correctly classified this acoustic energy as active sound. 2. **8.42s to 8.44s**: Energy plummeted from -40.3 dBFS to -51.9 dBFS. Physical silence began at **8.445s**. 3. **8.58s to 8.62s**: A 50 ms transient acoustic pulse occurred peaking at -29.8 dBFS (a mic pop). The detector's 80 ms minimum speech burst filter rejected this artifact, maintaining the gap. 4. **8.63s to 9.15s**: Pure ambient room silence held at -55.0 dBFS. 5. **9.16s**: Speaker phonated the next phrase, with energy rising to -32.7 dBFS. Physical silence ended at **9.160s**.

7.3 Investigation Conclusion & Scientific Decision

The detected duration of **0.715s** reflects the objective physical non-speech interval below -40 dBFS. The $\approx 0.65\text{s}$ difference from perceived pause is due to vocal breath sounds. The -40 dBFS threshold must **not** be raised prematurely: raising it to encompass breath sounds would cause false silence classifications across soft consonants and room noise. The user manually confirmed the listening comparison was accurate for this recording.

---

8. Scientific & Pedagogical Invariants: Avoiding Pseudoscientific Traps

1. **Acoustic vs Linguistic Separation**: Never conflate an acoustic metric (e.g. pause duration or speech activity ratio) with a linguistic metric (e.g. Word Error Rate) or semantic relevance.
2. **Measurement, Not Judgment**: Pauses are physical acoustic intervals. In professional presentations, deliberate pauses convey poise and emphasis. Never label all pauses as "mistakes" or "hesitations".
3. **No Fabricated Fluency Scores**: Do not generate fake composite scores (e.g. "Fluency: 84/100") without an empirically validated, published model trained on labeled speech corpora.
4. **Approximate Alignment Integrity**: Whisper word timestamps derive from encoder-decoder cross-attention weights, not acoustic HMM-GMM forced aligners. They provide approximate word-level navigation, not phoneme-level ground truth.

---

9. Pretrained Foundation ASR vs Future Trained Acoustic Fluency Model

In a viva defense, examiners frequently ask:

> *"If you are using OpenAI Whisper, where is your own AI contribution?"*

This table articulates the technical distinction:

Dimension	Pretrained Foundation ASR (Whisper)	Our Future Trained Fluency Model (Milestone 4)
Model Type	Encoder-Decoder Sequence-to-Sequence Transformer	Custom Supervised Regression Model (Ridge / GBDT / MLP)
Input	Log-mel spectrogram ($80\text{ channels} \times \text{frames}$)	Handcrafted Acoustic Feature Vectors (DSP Extracted)
Task	Transcribe acoustic waveform into text tokens	Predict continuous fluency ratings ($1.0 - 5.0$)
Features Used	Latent learned representations	F0 pitch variation, spectral flux, pause ratios, MFCCs
Training Data	680,000 hours of internet speech audio	Labeled corpora (Speechocean762 / LibriSpeech)
Role in Project	Speech-to-text pipeline & word alignment	**Core Project Contribution**: Algorithmic scoring of delivery

Whisper provides the linguistic transcript; our custom feature extractor and regression models evaluate delivery quality.

---

10. Security Defenses, Boundary Guards & Production Limitations

1. **Loopback Host Enforcement**: Endpoints reject requests from non-loopback IP addresses (`403 Forbidden: FORBIDDEN_ORIGIN`).
2. **Development Environment Guard**: Rejects requests if `ENVIRONMENT ≠ "development"` (`403 Forbidden: DEVELOPMENT_ONLY_ENDPOINT`).
3. **Session Traversal Prevention**: Sanitizes `session_id` using regex `^[a-zA-Z0-9_\-]+$` . Prevents `../../etc/passwd` directory traversal attacks.
4. **Private Storage & Git Hygiene**: Recordings reside in `backend/storage/recordings/` , strictly excluded via `.gitignore` . Raw audio and user transcripts are never committed to version control.
5. **No Cloud Leakage**: All PyAV audio decoding, NumPy frame RMS energy calculations, and Whisper inference execute 100% locally on CPU without external API calls.

---

11. Automated Verification Evidence & Comprehensive Test Matrix

Across the full stack, **132 automated tests pass with 100% success rate**:

11.1 Backend Test Suite (81 Tests Passed)

```
tests/test_acoustics.py ..............  (12 passed)
tests/test_acoustics_api.py ..........  (8 passed)
tests/test_audio_decoder.py ..........  (5 passed)
tests/test_audio_validator.py ........  (6 passed)
tests/test_cors.py ...................  (3 passed)
tests/test_errors.py .................  (2 passed)
tests/test_health.py .................  (1 passed)
tests/test_model_comparison.py .......  (10 passed)
tests/test_recordings.py .............  (12 passed)
tests/test_storage.py ................  (7 passed)
tests/test_transcription_api.py ......  (8 passed)
tests/test_transcription_service.py ..  (7 passed)
═══════════════════ 81 passed in 2.78s ═══════════════════
```

11.2 Frontend Test Suite (51 Tests Passed)

```
src/hooks/useAcoustics.test.ts (6 passed)
src/hooks/useAudioUpload.test.ts (7 passed)
src/hooks/useAudioRecorder.test.ts (9 passed)
src/data/exercises.test.ts (3 passed)
src/components/Header.test.tsx (3 passed)
src/hooks/useTranscription.test.ts (6 passed)
src/components/RecorderControls.test.tsx (4 passed)
src/components/ExerciseSelector.test.tsx (3 passed)
src/components/AudioPlayer.test.tsx (10 passed)
═══════════════════ 51 passed in 8.85s ═══════════════════
```

11.3 Production Build Validation

- `tsc -b && vite build` completed in **468ms** with zero TypeScript or bundling errors.

---

12. 22 Deep Technical Viva Defense Questions & Answers

Q1: What physical acoustic feature does your pause detection engine compute?

Answer: It computes Short-Time Energy Root Mean Square (RMS) over 25.0 ms windows (400 samples at 16 kHz) with a 10.0 ms hop (160 samples), converted to logarithmic decibels full scale ($\text{dBFS} = 20 \log_{10}(\text{RMS} + 10^{-7})$).

Q2: Why is the speech threshold set to -40.0 dBFS instead of 0 dBFS or -20 dBFS?

Answer: 0 dBFS is digital maximum (clipping). In standard room recordings, conversational voice peaks between -28 dBFS and -16 dBFS, while ambient noise floors sit between -55 dBFS and -50 dBFS. Setting the threshold at -40.0 dBFS provides $\approx 10\text{ dB}$ of headroom above background noise while capturing soft vowel tails.

Q3: What is the purpose of the 150 ms hangover window?

Answer: In English speech, unvoiced stop consonants (/p/, /t/, /k/) feature vocal tract closure intervals of 30–80 ms of pure acoustic silence before release. Without a hangover window, a single word would be fragmented into multiple disjoint speech bursts. The 150 ms hangover bridges these natural closures.

Q4: Why are pauses under 250 ms excluded from the internal gaps list?

Answer: Based on Goldman-Eisler's (1968) seminal psycholinguistic research, silent intervals under 250 ms represent articulatory motor execution (stop closures, consonant transitions), not communicative pauses. Communicative and cognitive pauses naturally exceed 250 ms.

Q5: Why do you separate leading and trailing silence from internal gaps?

Answer: Leading silence represents user preparation after clicking "Record"; trailing silence represents latency before clicking "Stop". Conflating these peripheral delays with internal pauses would heavily distort both speech activity ratios and pause duration statistics.

Q6: What is the difference between Overall Speaking Pace and Articulation Rate?

Answer: Overall Pace is total words divided by total recording duration (including pauses). Articulation Rate is total words divided by active speech duration only. Comparing both reveals whether a low pace is caused by slow articulation or long cognitive pauses.

Q7: Why is speaking rate labeled as `rate_is_transcript_dependent`?

Answer: Because acoustic analysis cannot identify words—it only measures energy. Calculating words per minute requires a recognized word count from the transcription engine.

Q8: How does clickable gap playback work without creating overlapping audio?

Answer: Both gap seeking and transcript word seeking manipulate the single existing HTML5 `<audio>` element reference (`audioRef.current.currentTime = seekTarget`). Because no secondary audio nodes or Web Audio contexts are spawned, overlapping playback is physically impossible.

Q9: Why seek to 0.75 seconds before a gap instead of the exact gap start?

Answer: Hearing silence in isolation provides no diagnostic value to a learner. Seeking 750 ms before the gap onset lets the user hear the preceding spoken phrase and perceive the pause in its natural context.

Q10: Why did you implement integer-hundredths arithmetic for decimal timestamps?

Answer: To eliminate IEEE 754 floating-point rounding errors (e.g. `9.155 * 100 = 915.4999999999999`). Converting to integer hundredths via `Math.round(seconds * 100)` guarantees clean, exact two-decimal timestamps (`mm:ss.SS`).

Q11: How does interactive transcript playback work?

Answer: When Whisper generates word timestamps, each word renders as a `<button>`. Clicking a word seeks the player to 0.25s before that word (`Math.max(0, start - 0.25)`) and triggers `play()`. The 0.25s lead-in captures the initial consonant burst.

Q12: How is real-time transcript highlighting synchronized?

Answer: The audio element's native `onTimeUpdate` event updates the `currentTime` state. Words whose `[start, end]` interval encompasses `currentTime` dynamically receive `.transcript-word-active`.

Q13: What happens if Whisper does not provide word-level timestamps?

Answer: The UI gracefully falls back: if words are missing, it renders clickable segment phrases; if segments are missing, it renders plain text `<p>`.

Q14: Why do you display an "Approximate Alignment" badge?

Answer: Whisper word timestamps are generated from encoder-decoder cross-attention weights, not acoustic forced aligners. Honesty requires labeling alignments as approximate navigation aids rather than claiming phoneme-level ground truth.

Q15: Why is audio processing executed via `asyncio.to_thread` in FastAPI?

Answer: PyAV decoding and NumPy frame RMS calculations are CPU-bound. Executing them directly in an async route would block Python's asyncio event loop, freezing concurrent API requests and health monitoring probes.

Q16: How do you prevent stale asynchronous responses when a user records a new take?

Answer: In `useAcoustics.ts`, `activeTakeRef` stores the current take number. When a fetch response resolves, it verifies `activeTakeRef.current === takeNumber`; if the user switched takes while analysis was in flight, the late response is discarded.

Q17: Why do you not generate an overall Fluency Score (e.g. 85/100) today?

Answer: Generating an arbitrary score without a trained, validated model would violate academic integrity. Milestone 4 will train a proper regression model on labeled datasets (Speechocean762).

Q18: In take_2.webm, why did the detector report a 0.715s gap when the user perceived ~1.4s?

Answer: Because between 7.76s (lexical word end) and 8.41s, the user released post-vocalic breath with energy between -27 dBFS and -24 dBFS, which is acoustically active. The physical silence below -40 dBFS began at 8.445s.

Q19: Why was an 80 ms minimum speech burst filter implemented?

Answer: To reject transient non-speech acoustic artifacts, such as microphone pops or clicks (which typically last 20–50 ms), preventing them from splitting true pauses into multiple short gaps.

Q20: What security measures protect user audio recordings?

Answer: Recordings are stored in Git-ignored private disk storage (`backend/storage/recordings/`), never exposed via static URLs, validated against path traversal (`^[a-zA-Z0-9_\-]+$`), and restricted to loopback and development environments.

Q21: What is the Word Error Rate (WER) formula?

Answer: $\text{WER} = \frac{S + D + I}{N}$, where S is substitutions, D is deletions, I is insertions, and N is the number of words in the ground-truth reference transcript, aligned via Levenshtein dynamic programming.

Q22: Why did we leave the ASR benchmark evaluation incomplete today?

Answer: Because verified reference transcripts for Clip 2 and Clip 3 have not yet been supplied. In accordance with academic integrity, reference text and WER scores must never be fabricated.

---

13. Live Evaluation Demo Script & Conference Paper Roadmap

13.1 Live Demonstration Steps for Examiners

1. **Launch Stack**: Verify FastAPI is healthy on `http://127.0.0.1:8000/api/health` and React frontend is running at `http://localhost:5173`.
2. **Audio Capture**: Select "Self-Introduction (Technical)", record a 15–20s response, and click Stop.
3. **Audio Upload**: Click "Upload Recording"; observe instant magic-byte validation and atomic disk persistence.
4. **Offline Speech Recognition**: Click "Transcribe Speech"; observe local Whisper CPU inference, word timestamp extraction, and safe text presentation with Academic Notice.
5. **Acoustic Delivery Analysis**: Click "Analyze Delivery"; observe off-event-loop execution and instant rendering of the **Audio Delivery Measurements** card.
6. **Delivery Metrics Review**: Point out duration, speech activity ratio (75%–90%), leading silence, and dual WPM (Total Pace vs Articulation Rate).
7. **Clickable Gap Seeking Demo**: Expand "View Internal Gaps Breakdown". Show decimal timestamps (`mm:ss.SS`). Click a gap button; verify it seeks to 0.75s before gap onset and plays.
8. **Interactive Transcript Demo**: Click a word in the transcript; verify it seeks to 0.25s before word onset and plays. Observe the synchronized real-time highlight moving across words.
9. **State Isolation Demo**: Click "Record Another Take"; observe that all previous measurements, transcripts, and highlights unmount and clear immediately.

13.2 Conference Paper Roadmap

- **Proposed Paper Title**: "*VERBAMIND: Evidence-Linked Acoustic Delivery and Speech Analytics for Technical Placement Communication Coaching*".
- **Key Empirical Contribution**: Decoupling acoustic delivery metrics (energy-thresholded pause distribution, articulation pace) from linguistic ASR token error rates, eliminating scoring biases against non-native technical graduates.